

```
# Boston Housing Price Prediction
```

```
import numpy as np
```

```
import pandas as pd
```

```
import torch
```

```
from torch import nn
```

```
from torch.utils.data import TensorDataset, DataLoader
```

```
from pathlib import Path
```

```
from sklearn.datasets import fetch_openml
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

```
import sys
```

```
print("Libraries imported successfully.")
```

```
# Load the data
```

```
print("Fetching Boston Housing dataset from OpenML...")
```

```
boston = fetch_openml(name='boston', version=1, as_frame=True, parser='auto')
```

```
X, y = boston.data, boston.target.astype(np.float32)
```

```
# Ensure all features are numeric and handle missing values
```

```
X = X.apply(pd.to_numeric, errors='coerce')
```

```
if X.isnull().values.any():
```

```
    X = X.fillna(X.median())
```

```
    print("Handled missing values.")
```

```
X_train_all, X_test, y_train_all, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
feature_scaler = StandardScaler()
x_train_scaled = feature_scaler.fit_transform(X_train_all)
x_test_scaled = feature_scaler.transform(X_test)
```

```
y_train = y_train_all.to_numpy(dtype=np.float32).reshape(-1, 1)
y_test = y_test.to_numpy(dtype=np.float32).reshape(-1, 1)
```

```
print(f"Training samples: {x_train_scaled.shape[0]}")
```

```
print(f"Test samples: {x_test_scaled.shape[0]}")
```

Define the Model

```
class BostonRegressionNet(nn.Module):
```

```
    def __init__(self, input_dim: int):
```

```
        super().__init__()
```

```
        self.network = nn.Sequential(
```

```
            nn.Linear(input_dim, 64),
```

```
            nn.ReLU(),
```

```
            nn.Linear(64, 32),
```

```
            nn.ReLU(),
```

```
            nn.Linear(32, 16),
```

```
            nn.ReLU(),
```

```
            nn.Linear(16, 1)
```

```
        )
```

```

def forward(self, x):
    return self.network(x)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = BostonRegressionNet(x_train_scaled.shape[1]).to(device)
print(f"Model initialized on {device}.")

# Training setup
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.005)

tensor_x_train = torch.tensor(x_train_scaled, dtype=torch.float32).to(device)
tensor_y_train = torch.tensor(y_train, dtype=torch.float32).to(device)
train_dataset = TensorDataset(tensor_x_train, tensor_y_train)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

num_epochs = 200
best_loss = float('inf')
patience = 15
epochs_no_improve = 0
best_state = None

print("Starting training...")
for epoch in range(1, num_epochs + 1):
    model.train()
    epoch_losses = []

```

```

for batch_x, batch_y in train_loader:
    optimizer.zero_grad()
    preds = model(batch_x)
    loss = criterion(preds, batch_y)
    loss.backward()
    optimizer.step()
    epoch_losses.append(loss.item())

epoch_loss = np.mean(epoch_losses)

if epoch % 20 == 0 or epoch == 1:
    print(f'Epoch {epoch:03d} | Train MSE: {epoch_loss:.4f}')

if epoch_loss < best_loss:
    best_loss = epoch_loss
    epochs_no_improve = 0
    best_state = model.state_dict()
else:
    epochs_no_improve += 1
    if epochs_no_improve >= patience:
        print(f'Early stopping triggered at epoch {epoch}')
        break

if best_state is not None:
    model.load_state_dict(best_state)

print("Training complete.")

```

```

# Evaluation

model.eval()

tensor_x_test = torch.tensor(x_test_scaled, dtype=torch.float32).to(device)

with torch.no_grad():

    y_pred = model(tensor_x_test).cpu().numpy().flatten()


y_test_flat = y_test.flatten()

mse = mean_squared_error(y_test_flat, y_pred)

mae = mean_absolute_error(y_test_flat, y_pred)

r2 = r2_score(y_test_flat, y_pred)


print('\nBoston Housing Price Prediction Results')

print('-----')

print(f'Test MSE : {mse:.3f}')

print(f'Test MAE : {mae:.3f}')

print(f'Test R2 : {r2:.3f}')


print('\nSample predictions (Actual vs Predicted):')

for actual, predicted in list(zip(y_test_flat[:10], y_pred[:10])):

    print(f'Actual: {actual:.1f}, Predicted: {predicted:.1f}')

```

Boston Housing Price Prediction Results

Test MSE : 12.252

Test MAE : 2.313

Test R2 : 0.833

Sample predictions (Actual vs Predicted):

Actual: 23.6, Predicted: 25.8

Actual: 32.4, Predicted: 38.6

Actual: 13.6, Predicted: 15.1

Actual: 22.8, Predicted: 24.9

Actual: 16.1, Predicted: 16.3

Actual: 20.0, Predicted: 21.9

Actual: 17.8, Predicted: 19.3

Actual: 14.0, Predicted: 15.0

Actual: 19.6, Predicted: 24.0

Actual: 16.8, Predicted: 20.3